

Title	AXI2APB Bridge
Project	None
Description	AXI2APB Bridge의 설계와 동작
Author	holelee
Date	2/May/2005

(*) AXI2APB Bridge

이제까지 팀내의 모든 IP들은 register setting이나 간단한 read/write의 경우 APB를 사용하였다. AXI BUS를 사용하게 되었다고 해당 부분을 모두 AXI Slave로 바꾸는 것은 말도 안되므로 AXI master가 BUS를 통해서 APB transaction을 만들 수 있도록 해야 한다. AHB를 사용할 때도 AHB2APB Interface를 사용한 것처럼 AXI slave로 APB를 통해 read/write를 수행하는 block을 만들었는데 AXI2APB Bridge이다. 사실 APB는 BUS라기 보다는 Interface에 가까우므로 Bridge라는 단어가 어울리지는 않지만 그냥 사용하기로 한다.

(*) AXI2APB Bridge의 제한 사항

AXI2APB Bridge는 일반적인 AXI Slave이므로 AXI의 모든 transaction을 받을 수 있다. 모든 type의 burst에 대해서 대응한다. 사실 APB에 붙어 있는 device에 Burst read/write를 사용할 가능성은 극히 낮다. 굳이 찾자면 FIFO를 읽기 위한 FIXED Burst가 사용될 가능성이 있을 뿐이다. 하지만 우선은 만드는 김에 INCR/WRAP burst도 지원되도록 만들었다.

AXI Slave로서 가지는 제한 점은 두 가지가 있는데, 첫째는 data width가 32bit으로 고정되어 있고 ARSIZE/AWSIZE로 WORD만 허용한다는 점이다. 일반적으로 APB는 32bit width의 data bus를 사용하기 때문인데 AXI도 32bit data width를 사용할 경우에는 문제가 없지만, AXI BUS가 64bit 인 경우에는 추후에 설계될 64bit-32bit AXI Bridge와 같은 것을 이용해서 해결해야 한다.

또 한가지 제한점은WSTRB 신호를 무시한다는 점인데, 이 부분은 어떻게 해결할 방법은 없다. APB에는 ByteEnable에 관련된 어떤 신호도 없기 때문이다. 이 부분은 AXI2APB Bridge를 access하는 AXI master가 올바른 request를 보내주기를 바라는 수 밖에 없다.

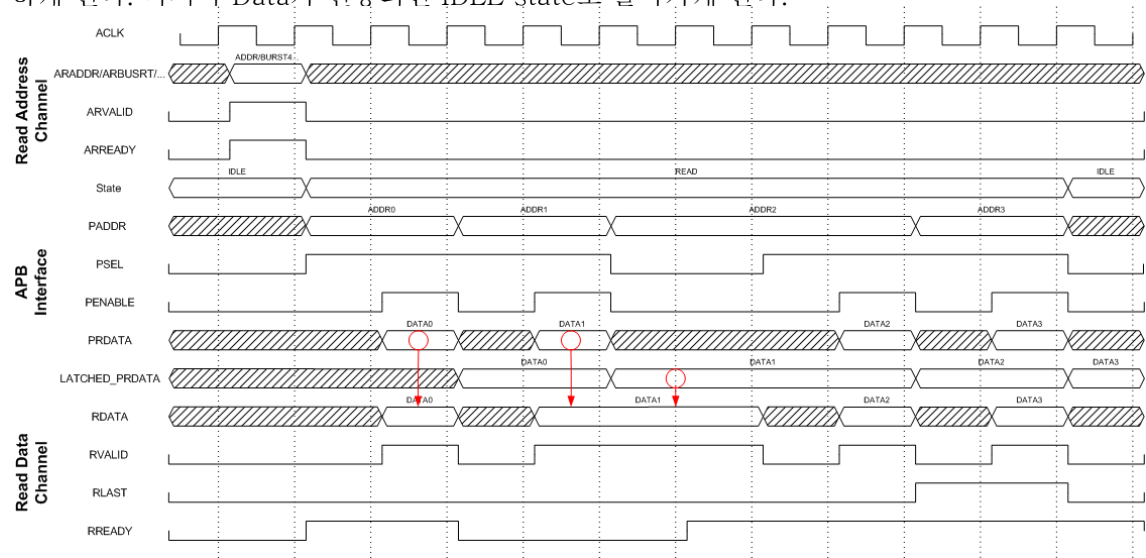
(*) AXI2APB Bridge의 동작

- Arbitration

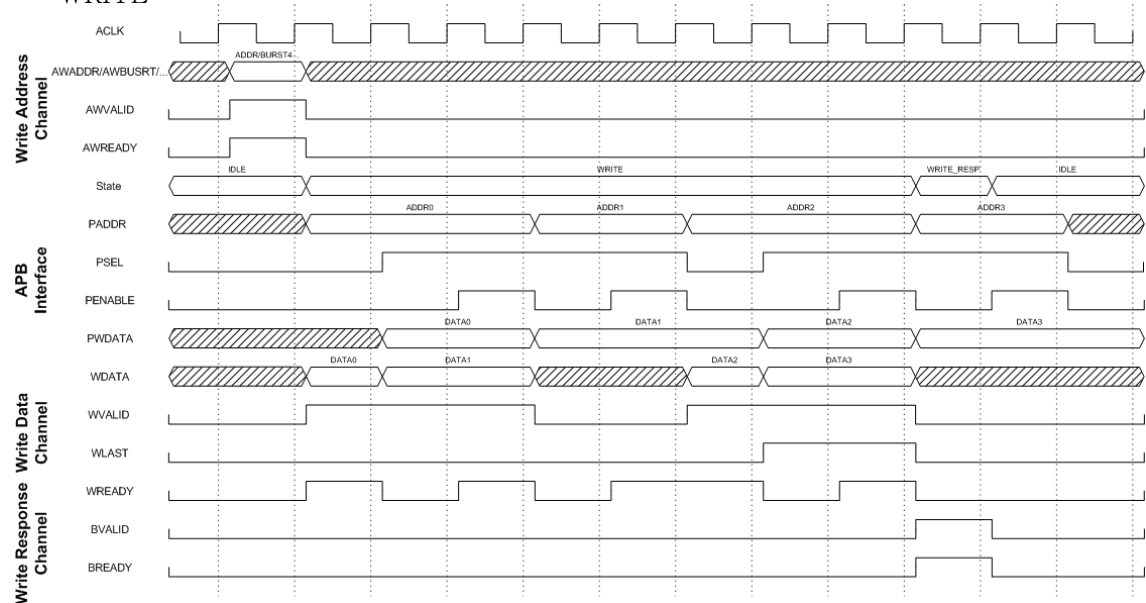
READ와 WRITE의 arbitration은 READ가 default인 round-robin 방식으로 수행한다. 기본적으로 ARREADY와 AWREADY는 ARVALID와 AWVALID와 combinational하게 연결되면 안되므로 1 cycle의 delay를 가질 수 있고, round-robin toggle이 항상 IDLE state에서 이루어지게 되어, maximum 2 cycle의 delay를 가지게 된다. 즉 read request가 연속적으로 들어오는 경우 첫번째 read request를 모두 처리하고 IDLE state가 되어 1 cycle delay를 가지고 round-robin state를 toggle하는데 다시 1 cycle의 delay를 가지게 되어 총 2 cycle의 delay를 가진다. 보통은 APB access가 많이 일어나지 않으므로 큰 문제가 되지 않을 것으로 본다.

- READ

READ는 다음과 같이 동작한다. 기본적으로 Read Address Channel을 통해 request가 들어오면 APB Bus쪽에 PSEL과 PENABLE을 assert해 주게 된다. PRDATA를 latch할 수 있을 때 RDATA에 PRDATA를 출력해 주고 RVALID가 assert 되는데, 이 때 RREADY가 high라면 다음 address로 진행하고, 그렇지 않으면 PRDATA를 latch하여 RDATA쪽에 값을 유지시켜 주면서 RREADY를 기다리게 된다. RREADY가 high가 되면 다음 address에서 data를 읽기 시작하게 된다. 마지막 Data가 전송되면 IDLE state로 돌아가게 된다.



- WRITE



WRITE의 동작은 위와 같다. Write Address channel의 request를 accept한 후에 첫번째

WDATA를 기다리게 된다. WDATA를 읽게 되면 APB쪽에 PSEL과 PENABLE을 assert하여 해당 data를 write를 수행하는 동시에 다음 WDATA를 latch한다. 만약 다음 WDATA가 available 하지 않으면(즉 WVALID가 low로 유지), WVALID가 high가 될 때까지 대기하게 된다. 마지막 WDATA가 latch되면 Write Response를 보내고 IDLE 상태로 복귀하게 된다.

- Response

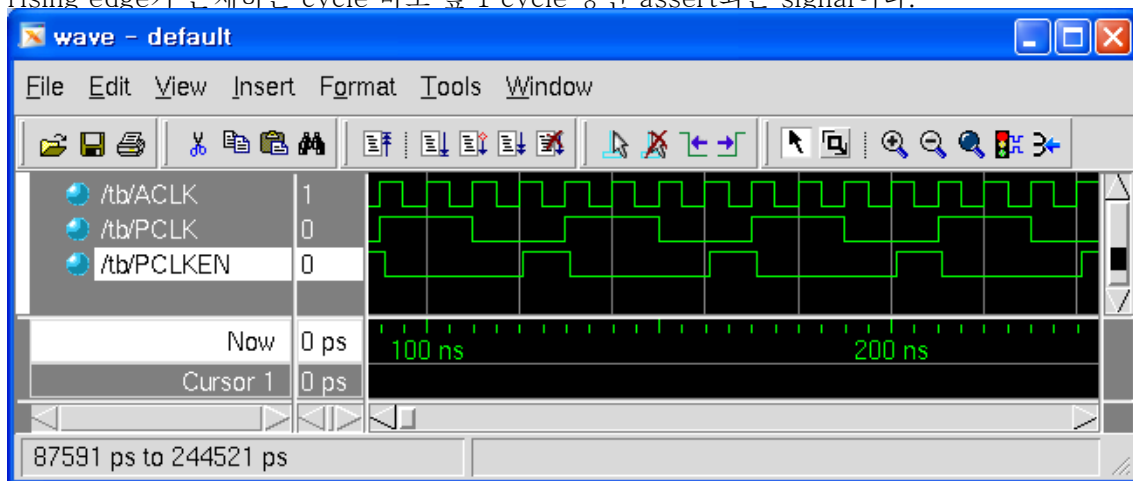
Read Response와 Write Response는 모두 OKAY를 assign하였다. ARM에서 제공하는 APB Bridge는 TrustZone과 같은 device를 위해서 slave error를 return하거나 decode error를 return하게 되는데, 설계된 AXI2APB Bridge는 그것을 지원하지 않는다.

(*) 세가지 버전

AXI2APB Bridge는 크게 세가지 버전이 존재한다.

- Full Version(AXI2APBBridge.v)

Full Version은 가장 복잡하게 구현된 버전으로, 우선 ACLK(AXI clock)과 PCLK(APB clock)이 서로 달라도 되고 AMBA3 APB에서 등장한 PREADY 신호를 지원한다. PCLK은 ACLK의 정수배의 주기를 가지고 PCLK의 rising edge가 ACLK의 rising edge에 맞추어져 있다는 것을 가정한다. AXI2APB Bridge 자체는 PCLK을 사용하지 않고, PCLK의 rising edge를 detect할 수 있는 PCLKEN이라는 signal을 입력으로 받게 된다. PCLKEN은 아래 그림과 같이 PCLK의 rising edge가 존재하는 cycle 바로 앞 1 cycle 동안 assert되는 signal이다.



PREADY 신호는 AMBA 3.0의 APB에 등장하여 APB slave 입장에서 flow control을 할 수 있는 신호이다. APB slave는 PREADY를 low로 하여 transaction을 delay시킬 수 있는데, PSEL과 PENABLE이 high이고 PREADY가 high라면 transaction이 종료되고, PREADY가 low라면 PSEL과 PENABLE이 계속 high로 유지되게 된다. 가변적인 delay를 가지는 APB slave를 설계하여 attach할 수 있다.

- PREADY Version(AXI2APBBridge_PREADY.v)

PREADY Version은 PREADY 신호를 지원하기는 하지만 ACLK과 PCLK이 완전히 동일하다는 가정하에서 만들어진 버전이다. PREADY 신호를 지원하므로 가변 delay를 가지는 APB slave를 사용할 수 있지만, PCLK이 ACLK과 동일하게 동작하므로 많은 slave를 붙이게 되면 mux delay로 인해 ACLK을 높일 수 없을 가능성이 있다.

- Simple Version(AXI2APBBridge_Simple.v)

Simple Version은 PCLK이 ACLK과 동일하고 APB Bus도 AMBA 2.0 Spec에 준하여 개발되었다. PREADY 신호가 없으므로 APB를 통한 READ/WRITE는 2 cycle 걸쳐서 일어나게 된다.

(*) AXI2APB Bridge의 사용법

AXI2APB Bridge는 위에서 언급된 3가지 버전에 따라서 3개의 Verilog Source Code(각각, AXI2APBBridge.v AXI2APBBridge_PREADY.v AXI2APBBridge_Simple.v)로 구성된다. 원하는 Spec에 맞추어 적절한 Verilog Code를 선택하여야 한다. 그 다음에 사용자가 원하는 APB Slave 갯수에 맞추어 Verilog Code를 변경해야 한다. 설계된 Verilog Code는 우선 총 4개의 APB slave를 지원하도록 되어 있다. 따라서 PSEL, PRDATA, PREADY를 각각 4개씩 가지고 있다. 실제 사용을 위해서는 그것을 늘리기 위해서 직접 Verilog code를 editing하여야 하는데 Verilog code 끝부분에 address decoding을 수행하여 PSEL을 생성하는 부분, PRDATA/PREADY mux가 모여 있어 비교적 쉽게 변경할 수 있다.